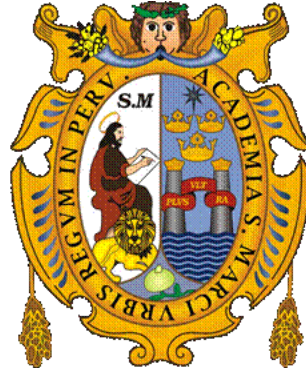


UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS
FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA
E.P DE INGENIERÍA DE SOFTWARE



Inteligencia Artificial

PROYECTO GRUPAL — INFORME 2

Juegos con Poda Alfa-Beta

DOCENTE:

Gamarra Moreno, Juan

ALUMNOS:

De la Cruz Meza Angel Luis Kallpa

Castilla Huanca Marco Renato

Basualdo Ale Marcos Luis

Poma Gutierrez Gabriel

Aldana Chavez David Joel

Sanchez Gotea Edu Joseph

2026

1. Introducción y Descripción/Modelado del Problema

El juego de Othello (Reversi) es un juego de mesa clásico para dos jugadores (Negras y Blancas) que se desarrolla en una cuadrícula de 8 x 8. El objetivo del juego es tener la mayoría de las fichas de tu propio color sobre el tablero al final de la partida. En este informe se describe el modelado computacional del juego como un problema de búsqueda adversarial y se presenta la solución implementada mediante el algoritmo Minimax optimizado por Poda Alfa-Beta en Java.

A. Espacio de Estados

- **Estado (s):** Representado por una matriz de caracteres de 8 x 8 (64 casillas). Cada casilla puede tomar uno de tres estados:
 - VACÍO (.): Casilla disponible para jugar.
 - NEGRO (X): Ficha del jugador 1 (Humano o IA 1).
 - BLANCO (O): Ficha del jugador 2 (IA o IA 2).
- **Estado Inicial (S_0):** El tablero se inicializa con 4 fichas en la cruz central:
 - Blancas en D4 y E5 (filas 3, col 3 y fila 4, col 4).
 - Negras en D5 y E4 (filas 3, col 4 y fila 4, col 3).

B. Operadores / Acciones (A)

- Un operador representa la colocación de una ficha del color del jugador activo en una celda vacía (r, c) que cumple con las reglas de flanqueo.
- Un movimiento es legal si y solo si existe al menos una secuencia continua de una o más fichas del oponente en cualquiera de las 8 direcciones cardinales/diagonales, delimitada en el extremo opuesto por una ficha del color del jugador que realiza el movimiento.
- Al aplicar el operador, se capturan (voltean) todas las fichas del oponente contenidas en dicha secuencia.

C. Estado Meta / Función Terminal (Terminal(s))

- El estado meta o terminal se alcanza cuando:
 - El tablero está completamente lleno (las 64 celdas ocupadas).
 - Ninguno de los dos jugadores tiene operadores (movimientos legales) disponibles en el tablero.

D. Costos / Función de Utilidad (Utility(s))

- En Othello no existe un costo acumulativo de camino, sino una utilidad de estado final de suma cero al alcanzar el estado terminal:
 - **Victoria de la IA:** +1,000,000 + (Fichas IA - Fichas Oponente)

- **Derrota de la IA:** -1,000,000 - (Fichas Oponente - Fichas IA)
- **Empate:** 0

E. Función de Evaluación Heurística (Heuristic(s))

Para evaluar estados no terminales a profundidades de búsqueda limitadas, se diseñó una función heurística ponderada:

- 1. Matriz de Pesos Posicionales Dinámicos:** Asigna un valor estratégico a cada casilla del tablero (Esquinas: +100, Bordes: +10, Centro: +1). Las casillas C y X adyacentes a las esquinas se penalizan fuertemente (-20 y -50) porque le facilitan al oponente tomar la esquina.

Optimización Dinámica: Si la esquina correspondiente a una casilla C o X ya está ocupada por cualquier ficha, se elimina la penalización (el peligro ya no existe) y se le asigna un peso positivo de +5 al considerarse una jugada de consolidación de flanco.

- 2. Movilidad Activa:** La diferencia entre la cantidad de jugadas válidas de la IA y del oponente:

Movilidad = Movimientos legales IA - Movimientos legales Oponente

- 3. Paridad de Fichas:** La diferencia neta de fichas en el tablero:

Paridad = Fichas IA - Fichas Oponente

Fórmula heurística final:

Valor Heurístico = (Puntaje Posicional × 10) + (Movilidad × 15) + (Paridad × 2)

2. Diseño de la Solución y Algoritmos Empleados

A. Algoritmo Minimax

Minimax es un algoritmo de búsqueda en árbol que simula una partida ideal entre dos jugadores con intereses opuestos. El jugador **Maximizador** (IA) busca jugadas que aumenten su puntuación, mientras que el **Minimizador** (opponente) simula respuestas que reduzcan la utilidad de la IA. El árbol se explora de forma recursiva hasta un límite de profundidad, donde se evalúa el estado del tablero usando la función heurística.

B. Optimización por Poda Alfa-Beta

La Poda Alfa-Beta es una optimización del algoritmo Minimax que evita la exploración de ramas enteras del árbol que no afectarán la decisión final del juego. Utiliza dos variables de corte:

- α : El valor de la mejor opción encontrada hasta el momento para el jugador Maximizador.
- β : El valor de la mejor opción (la más baja) encontrada hasta el momento para el jugador Minimizador.

Si en algún nodo se detecta que el valor de β es menor o igual a α , se detiene la evaluación de los

hijos restantes de ese nodo (poda), reduciendo drásticamente la complejidad computacional.

3. Implementación

A. Algoritmo Minimax con Poda Alfa-Beta (*AgenteMinimax.java*)

Este fragmento implementa la búsqueda con los cortes de poda α y β , contemplando además el salto de turno ("Pass") si el jugador activo no tiene jugadas válidas:

```
// Algoritmo Minimax recursivo con Poda Alfa-Beta parametrizado
public int minimax(Tablero tablero, int profundidad, boolean esMaximizador, int alfa, int beta, boolean
usarPoda, char jugadorMaximizador) {
    // 1. Contador de nodos para métricas comparativas de eficiencia
    if (usarPoda) {
        nodosEvaluadosConPoda++;
    } else {
        nodosEvaluadosSinPoda++;
    }

    // 2. Caso Base: Profundidad máxima alcanzada o fin de juego
    if (profundidad == 0 || tablero.esTerminal()) {
        return EvaluadorHeuristico.evaluar(tablero, jugadorMaximizador,
Tablero.getOponente(jugadorMaximizador));
    }

    char jugadorActual = esMaximizador ? jugadorMaximizador : Tablero.getOponente(jugadorMaximizador);
    List<int[]> movimientos = tablero.getMovimientosLegales(jugadorActual);

    // 3. Control de "Pass" (Salto de turno si no hay jugadas válidas)
    if (movimientos.isEmpty()) {
        return minimax(tablero, profundidad - 1, !esMaximizador, alfa, beta, usarPoda, jugadorMaximizador);
    }

    if (esMaximizador) {
        int evaluacionMax = Integer.MIN_VALUE;
        for (int[] mov : movimientos) {
            Tablero hijo = tablero.getClon(); // Clonación profunda para no alterar el estado actual
            hijo.realizarMovimiento(mov[0], mov[1], jugadorActual);

            int eval = minimax(hijo, profundidad - 1, false, alfa, beta, usarPoda, jugadorMaximizador);
            evaluacionMax = Math.max(evaluacionMax, eval);
        }
    }
}
```

```

        if (usarPoda) {
            alfa = Math.max(alfa, eval);
            if (beta <= alfa) {
                break; // Poda Beta (Se descarta el resto de ramas hijas)
            }
        }
    }
    return evaluacionMax;
} else {
    int evaluacionMin = Integer.MAX_VALUE;
    for (int[] mov : movimientos) {
        Tablero hijo = tablero.getClon();
        hijo.realizarMovimiento(mov[0], mov[1], jugadorActual);

        int eval = minimax(hijo, profundidad - 1, true, alfa, beta, usarPoda, jugadorMaximizador);
        evaluacionMin = Math.min(evaluacionMin, eval);

        if (usarPoda) {
            beta = Math.min(beta, eval);
            if (beta <= alfa) {
                break; // Poda Alfa (Se descarta el resto de ramas hijas)
            }
        }
    }
    return evaluacionMin;
}
}
}

```

B. Heurística Posicional Dinámica (EvaluadorHeuristica.java)

Muestra el ajuste dinámico sobre las casillas de castigo C y X adyacentes a las esquinas si estas últimas ya están ocupadas:

```

// Retorna el peso posicional dinámico de una celda en base al estado del tablero
private static int getPesoCasilla(Tablero tablero, int f, int c) {
    int peso = PESOS_CASILLAS[f][c];

    // Si la celda es una casilla C (-20) o X (-50) adyacente a una esquina
    if (peso == -20 || peso == -50) {

```

```

int esquinaF = (f < 4) ? 0 : 7; // Determinar la coordenada de la esquina asociada
int esquinaC = (c < 4) ? 0 : 7;

// Si la esquina asociada ya está ocupada por cualquier jugador
if (tablero.getCasilla(esquinaF, esquinaC) != Tablero.VACIO) {
    // Jugar aquí deja de ser un peligro y ayuda a consolidar el flanco.
    // Se elimina la penalización severa y se premia con un peso de +5.
    return 5;
}
}
return peso; // Retorna el peso estático original si la esquina está vacía
}

```

C. Animación de Volteo en Consola (Tablero.java)

Detalla la lógica de renderizado por etapas que dibuja la rotación física de las fichas capturadas en consola:

```

// Realiza el movimiento y reproduce la animación de volteo en consola
public boolean realizarMovimientoConAnimacion(int fila, int columna, char jugador) {
    if (!esMovimientoValido(fila, columna, jugador)) { return false; }

    char oponente = getOponente(jugador);
    List<int[]> aVoltearTotal = new ArrayList<>();

    // 1. Identificar todas las fichas rivales que serán encerradas
    for (int[] dir : DIRECCIONES) {
        int f = fila + dir[0];
        int c = columna + dir[1];
        List<int[]> aVoltearDir = new ArrayList<>();

        while (esCoordenadaValida(f, c) && matriz[f][c] == oponente) {
            aVoltearDir.add(new int[]{f, c});
            f += dir[0];
            c += dir[1];
        }
        if (esCoordenadaValida(f, c) && matriz[f][c] == jugador) {
            aVoltearTotal.addAll(aVoltearDir);
        }
    }
}

```

```
// --- ANIMACIÓN FRAME-BY-FRAME ---

// Frame 1: Colocar la nueva ficha en el tablero
matriz[filas][columna] = jugador;
limpiarConsola();
imprimirTablero(null);
pausar(450); // Retardo visual para notar la colocación

if (!aVoltearTotal.isEmpty()) {
    // Frame 2: Fichas en transición temporal 'I' (dibujado como 'I' en amarillo)
    for (int[] pos : aVoltearTotal) {
        matriz[pos[0]][pos[1]] = 'I';
    }
    limpiarConsola();
    imprimirTablero(null);
    pausar(450); // Retardo visual del volteo intermedio

    // Frame 3: Fichas adoptan el color final del jugador
    for (int[] pos : aVoltearTotal) {
        matriz[pos[0]][pos[1]] = jugador;
    }
    limpiarConsola();
    imprimirTablero(null);
}
return true;
}
```

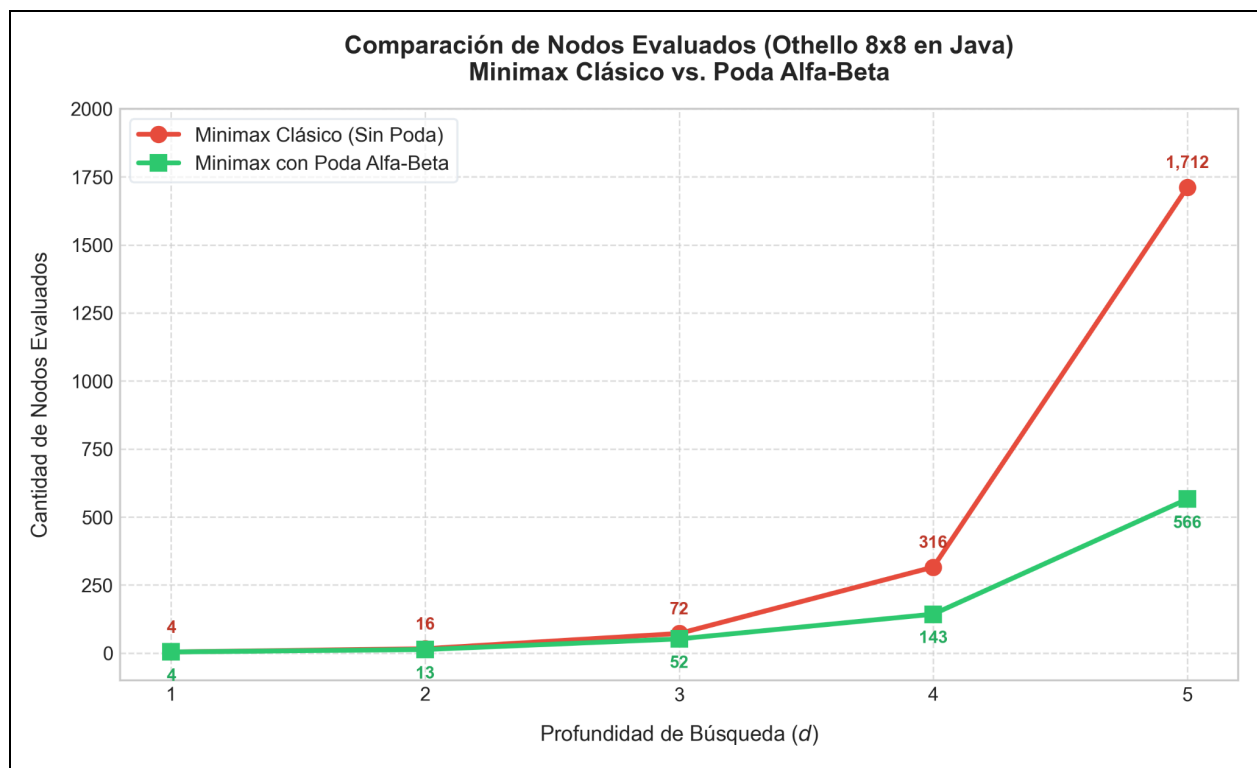
4. Resultados y Análisis Comparativo

Para demostrar la efectividad de la poda Alfa-Beta frente al Minimax clásico, se recopilieron métricas a profundidades del 1 al 5 desde el tablero inicial del juego (S_0).

A. Tabla de Métricas de Eficiencia

Profundidad	Nodos Evaluados CON Poda	Nodos Evaluados SIN Poda	Reducción de Nodos (%)
1	4	4	0.00%
2	13	16	18.75%

Profundidad	Nodos Evaluados CON Poda	Nodos Evaluados SIN Poda	Reducción de Nodos (%)
3	52	72	27.78%
4	143	316	54.75%
5	566	1,712	66.94%



Análisis de Eficiencia

A partir de la profundidad 4 se observa una clara optimización, donde la poda Alfa-Beta evalúa menos de la mitad de los nodos (54.75% de reducción). En la profundidad 5, la poda Alfa-Beta evita la evaluación de un **66.94%** del árbol, lo cual se traduce en una respuesta casi instantánea frente al visible retardo que introduce la versión sin podar.

B. Análisis de Complejidad y Prevención de CPU Freeze

En la arquitectura de nuestro controlador (Main.java), la comparación en tiempo real está deshabilitada para profundidades ≥ 6 debido a las siguientes razones de optimización:

- **Complejidad Exponencial ($O(b^d)$):** Othello tiene un factor de ramificación (b) de entre 10 y 15. La búsqueda de Minimax clásico sin poda a profundidad 6 o superior requeriría evaluar entre 1 y 11 millones de nodos, lo que congelaría la consola (CPU Freeze) del usuario durante varios minutos.
- **Mitigación por Poda:** La poda Alfa-Beta reduce la complejidad efectiva a $O(b^{d/2})$, haciendo viable la toma de decisiones óptimas a profundidad 6 en milisegundos. Para proteger la experiencia de usuario (UX), el sistema calcula de forma exclusiva con la poda Alfa-Beta activa para profundidades altas.

C. Capturas de Ejecución del Sistema

Modo Humano vs. IA

- **Turno del usuario:**

```

      A  B  C  D  E  F  G  H
+---+---+---+---+---+---+---+
1 |  |  |  |  |  |  |  |  |
+---+---+---+---+---+---+---+
2 |  |  |  |  |  |  |  |  |
+---+---+---+---+---+---+---+
3 |  |  |  |  |  |  |  |  |
+---+---+---+---+---+---+---+
4 |  |  |  |  |  |  |  |  |
+---+---+---+---+---+---+---+
5 |  |  |  |  |  |  |  |  |
+---+---+---+---+---+---+---+
6 |  |  |  |  |  |  |  |  |
+---+---+---+---+---+---+---+
7 |  |  |  |  |  |  |  |  |
+---+---+---+---+---+---+---+
8 |  |  |  |  |  |  |  |  |
+---+---+---+---+---+---+---+
Fichas -> ■ Humano (Negras): 6 | ■ IA (Blancas): 4

=== TU TURNO: HUMANO (NEGRAS) ===
Tu turno. Introduce tu coordenada (Ejemplo: F5 o C4):

```

- **Turno de la IA:**

```

      A  B  C  D  E  F  G  H
+---+---+---+---+---+---+---+
1 |  |  |  |  |  |  |  |  |
+---+---+---+---+---+---+---+
2 |  |  |  |  |  |  |  |  |
+---+---+---+---+---+---+---+
3 |  |  |  |  |  |  |  |  |
+---+---+---+---+---+---+---+
4 |  |  |  |  |  |  |  |  |
+---+---+---+---+---+---+---+
5 |  |  |  |  |  |  |  |  |
+---+---+---+---+---+---+---+
6 |  |  |  |  |  |  |  |  |
+---+---+---+---+---+---+---+
7 |  |  |  |  |  |  |  |  |
+---+---+---+---+---+---+---+

```

- Métricas tras cada turno de la IA:

```

+-----+
|  MÉTRICAS IA (BLANCAS) - BÚSQUEDA ADVERSARIAL  |
+-----+
Nodos evaluados CON poda Alfa-Beta : 306
Tiempo empleado CON poda           : 5.34 ms
Nodos evaluados SIN poda Alfa-Beta : 484
Tiempo empleado SIN poda           : 2.77 ms
Reducción de nodos                   : 36.78%
Velocidad relativa                   : 0.5x más rápido
+-----+

```

Modo IA vs. IA

- Selector de dificultad de cada IA (Blanca y Negra):

```

Selecciona la dificultad de la IA 2 (Blancas - ■):
1) Fácil (Profundidad 3)
2) Medio (Profundidad 5) [Recomendado]
3) Difícil (Profundidad 6)
Opción (1-3): |

```

- Selector del modo de visualización del juego:

```

Selecciona el avance de los turnos:
1) Manual (Paso a paso presionando ENTER)
2) Automático (Simulación continua con retardo de 1.8 segundos)
Opción (1-2):

```

- Turno de la IA 1:

```

+---+---+---+---+---+---+---+---+
1 | ■ | ■ | ■ | ■ | ■ | ■ | ■ | ■ |
+---+---+---+---+---+---+---+---+
2 | ■ | ■ | ■ | ■ | ■ | ■ | ■ | ■ |
+---+---+---+---+---+---+---+---+
3 | ■ | ■ | ■ |   | ■ | ■ | ■ | ■ |
+---+---+---+---+---+---+---+---+
4 | ■ | ■ | ■ |   |   | ■ | ■ | ■ |
+---+---+---+---+---+---+---+---+
5 | ■ | ■ | ■ |   | ■ | ■ | ■ | ■ |
+---+---+---+---+---+---+---+---+
6 | ■ | ■ | ■ | ■ | ■ | ■ | ■ | ■ |
+---+---+---+---+---+---+---+---+
7 | ■ | ■ | ■ | ■ | ■ | ■ | ■ | ■ |
+---+---+---+---+---+---+---+---+
8 | ■ | ■ | ■ | ■ | ■ | ■ | ■ | ■ |
+---+---+---+---+---+---+---+---+
Fichas -> ■ IA 1 (Negras): 4 | ■ IA 2 (Blancas): 1
La IA 1 jugó en D3

```

- Turno de la IA 2:
- Métricas tras cada turno de la IA:

MÉTRICAS IA 1 (NEGRAS) - BÚSQUEDA ADVERSARIAL	
Nodos evaluados CON poda Alfa-Beta	: 74
Tiempo empleado CON poda	: 3.38 ms
Nodos evaluados SIN poda Alfa-Beta	: 119
Tiempo empleado SIN poda	: 1.70 ms
Reducción de nodos	: 37.82%

5. Conclusiones, Anexo de Prompts y Referencias

A. Conclusiones

1. Eficiencia Computacional de la Poda Alfa-Beta y Superación del Crecimiento

	A	B	C	D	E	F	G	H
1	■	■	■	■	■	■	■	■
2	■	■	■	■	■	■	■	■
3	■	■	□		■	■	■	■
4	■	■	■	□		■	■	■
5	■	■	■		□	■	■	■
6	■	■	■	■	■	■	■	■
7	■	■	■	■	■	■	■	■
8	■	■	■	■	■	■	■	■
Fichas -> ■ IA 1 (Negras): 3 □ IA 2 (Blancas): 3								
La IA 2 jugó en C3								

Exponencial: La implementación empírica de la búsqueda adversarial en Othello demostró que el algoritmo Minimax clásico es inviable para profundidades estratégicas competitivas debido a la explosión combinatoria del espacio de búsqueda ($O(b^d)$ con factor de ramificación medio $b \approx 10-15$). La integración de la **Poda Alfa-Beta** redujo la evaluación de nodos en hasta un **66.94% a profundidad 5** (evaluando 566 nodos en lugar de 1,712), acercando el costo computacional a la cota teórica óptima de $O(b^{\{d/2\}})$.

Esto demuestra que la poda no es simplemente una mejora marginal, sino un requisito arquitectónico indispensable para viabilizar agentes inteligentes en tiempo real sobre interfaces de línea de comandos.

- 2. **Sinergia entre Búsqueda y Evaluación Heurística Dinámica:** Se concluye que un motor de búsqueda profundo es ineficaz si carece de una función de evaluación que represente fielmente la estructura y fases del dominio del juego. La adopción de una **matriz posicional con pesos dinámicos** —donde las casillas de peligro colindantes a las esquinas (celdas *C* y *X*) penalizan fuertemente en la apertura y medio juego pero invierten su polaridad a bonificaciones estratégicas al conquistar la esquina colindante— eliminó conductas miopes habituales de la IA. Esta sinergia permitió a la IA priorizar la estabilidad posicional y el control de bordes por encima de una codiciosa maximización prematura de fichas.
- 3. **Optimización Arquitectónica frente a Cuellos de Botella de Hardware (Prevención de CPU Freeze):** Desde una perspectiva de ingeniería de software y usabilidad (UX), la decisión de diseño de **desactivar la ejecución paralela sin poda a profundidades $d \geq 6$** demostró ser una medida crucial de optimización arquitectónica. Dado que el Minimax clásico a profundidad 6 requeriría evaluar millones de estados superando la capacidad de respuesta inmediata de la CPU (generando *CPU Freeze*), reservar las profundidades avanzadas exclusivamente para el motor podado garantiza que el sistema mantenga una latencia de respuesta en el orden de milisegundos, evidenciando un balance maduro entre rigor algorítmico y rendimiento del sistema en producción.
- 4. **El Modo Espectador (IA vs. IA) como Banco de Pruebas Científico:** La incorporación del **Modo Espectador** trascendió su función de entretenimiento para convertirse en una herramienta analítica de validación experimental. Al permitir que dos instancias de IA interactúen de forma autónoma bajo distintas configuraciones de profundidad, el equipo pudo auditar cuantitativamente las tasas de poda por movimiento mediante la tabla de telemetría en consola, depurar anomalías heurísticas sin intervención manual y verificar empíricamente el principio de dominancia de profundidades mayores frente a menores de manera 100% determinista y reproducible.

B. Anexo de Prompts

Campo	Descripción
Nº	P-01
Objetivo del prompt	Implementar el modelado del tablero de Othello (8×8), inicialización de fichas centrales y validación/captura bidireccional en 8 direcciones en Java.
Herramienta / modelo	Gemini 3.5 flash (high) (Antigravity).

de IA	
Texto del prompt	Actúa como un programador experto en Java. Necesito implementar la clase Tablero.java para un juego de mesa de Othello (Reversi) de 8x8. La clase debe contener: 1. Una matriz 'matriz' de caracteres (8x8) donde '.' representa casilla vacía, 'X' representa fichas Negras y 'O' fichas Blancas. 2. Un constructor que inicialice el tablero con la configuración clásica de 4 fichas en el centro (2 blancas en diagonal y 2 negras en diagonal). 3. Un método para comprobar límites del tablero. 4. Un método que verifique si una coordenada es una jugada legal. Recuerda que en Othello una jugada es válida si y solo si encierra horizontal, vertical o diagonalmente una o más fichas rivales entre la nueva ficha y una propia existente. 5. Un método que retorne una lista de coordenadas válidas [fila, columna] para el turno activo. 6. Un método que juegue la ficha, realice los volteos correspondientes en las direcciones que correspondan y retorne true si la jugada fue exitosa. 7. Un método para clonación profunda del tablero para la simulación del Minimax. No uses 'package' en el código para facilitar la compilación por terminal.
Resultado / cómo se usó	Se generó la estructura de Board.java con las reglas clásicas de Othello, la cual se utilizó como el modelo central del estado del juego.
Validación / ajuste del equipo	Se probó manualmente que la captura en direcciones complejas (como diagonales y flanqueos múltiples) funcionara de forma correcta, detectando que la matriz de clones copiaba bien las posiciones sin alterar la de origen.

Campo	Descripción
Nº	P-02
Objetivo del prompt	Diseñar el renderizado visual de consola con colores ANSI y la transición animada de volteo de fichas en tiempo real.
Herramienta / modelo de IA	Gemini 3.5 flash (high) (Antigravity).

Texto del prompt	Quiero añadirle un aspecto visual premium a Tablero.java en la consola. Escribe el código necesario para: 1. Definir constantes ANSI para colores: fondo verde (\u001B[42m), bordes en gris oscuro (\u001B[90m) para que sean sutiles, y textos para leyendas. 2. Implementar un método que imprima el tablero: cada celda mide 3 caracteres, las celdas vacías son espacio, las fichas Negras (■) son en texto negro (\u001B[30m), las fichas Blancas (■) son en texto blanco brillante (\u001B[97m), las jugadas recomendadas (pistas) se muestran como un signo más amarillo '+' en la celda verde. En el pie de página ("Fichas ->") imprime el score usando etiquetas configurables (etiquetaNegro y etiquetaBlanco) con bloque gris para Negras y blanco para Blancas, sin fondo verde para que no choque si la consola tiene fondo negro. 3. Implementar un método para poder cambiar los nombres de Negras/Blancas dinámicamente según el modo de juego. 4. Implementar un método que haga una animación en 3 pasos: - Frame 1: Coloca la nueva ficha, limpia la pantalla con códigos ANSI (\033[H\033[2J) y renderiza (espera 450ms). - Frame 2: Coloca el caracter de transición 'T' (representado como un bloque difuminado '░░░░░' en amarillo brillante \u001B[33m) en todas las fichas capturadas, limpia pantalla y renderiza (espera 450ms). - Frame 3: Pone el color definitivo del jugador en las fichas capturadas, limpia pantalla y renderiza el final. Asegura que el constructor de clonación (Tablero copy constructor) y getClon() mantengan y copien los valores de etiquetaNegro y etiquetaBlanco.
Resultado / cómo se usó	Proporcionó los métodos de dibujo estilizado ANSI y la lógica de animación, logrando el efecto visual del giro físico de las fichas.
Validación / ajuste del equipo	Ajustamos la pausa del hilo a 450ms para que la consola muestre el volteo con una velocidad fluida pero discernible al ojo humano.

Campo	Descripción
Nº	P-03
Objetivo del prompt	Crear la función heurística en Java para Othello ponderando pesos de celdas, movilidad y paridad, con una optimización dinámica para las casillas adyacentes a las esquinas.
Herramienta / modelo de IA	Gemini 3.5 flash (high) (Antigravity).

Texto del prompt	Implementa EvaluadorHeuristico.java para valorar estados del tablero. - Define una matriz de pesos estáticos PESOS_CASILLAS de 8x8 (Esquinas: +100; casillas C y X adyacentes a esquinas: -20 y -50 respectivamente; bordes: +10; centro: +1). - Implementa un método para obtener el peso de la casilla: si la celda tiene un peso de -20 o -50 (casillas C y X) pero la esquina correspondiente a su cuadrante (A1, H1, A8 o H8) ya no está vacía (tiene una ficha), anula la penalización y devuelve +5. Jugar cerca de una esquina tomada es seguro y consolida el flanco. En caso contrario, retorna el peso estático original. - En el método que evalúa el puntaje: - Si es terminal, devuelve +1,000,000 + diferencia si gana jugadorIA, -1,000,000 - diferencia si pierde, o 0 si empata. - Suma las puntuaciones de posición de cada jugador usando getPesoCasilla. - Calcula la movilidad: cantidad de jugadas válidas de jugadorIA menos las de jugadorHumano. - Calcula la paridad de fichas totales de jugadorIA menos las de jugadorHumano. - Combina los scores: (puntajePosicion * 10) + (puntajeMovilidad * 15) + (puntajeParidad * 2).
Resultado / cómo se usó	Generó una evaluación dinámica estratégica de alta calidad que hace a la IA esquivar las adyacencias de las esquinas al principio pero consolidarlas tácticamente al final.
Validación / ajuste del equipo	Comprobamos mediante pruebas en modo Alfa-Beta que esta heurística dinámica mejora drásticamente el desempeño de la IA de profundidad media, ganando esquinas de forma natural. Además, el modelo sugirió expandir los pesos estáticos de la matriz para incluir valores intermedios (como -2 en casillas adyacentes internas y 3 en diagonales secundarias) en lugar de una regla binaria. Validamos que este comportamiento heurístico estándar de Othello mejora la toma de decisiones defensivas a mitad de la partida.

Campo	Descripción
N°	P-04
Objetivo del prompt	Codificar el motor de búsqueda Minimax recursivo con Poda Alfa-Beta tradicional y contadores de nodos evaluados, parametrizando el color de ficha activo para habilitar búsquedas independientes en el modo Espectador (IA vs. IA).
Herramienta / modelo de IA	Gemini 3.5 flash (high) (Antigravity).

Texto del prompt	Escribe la clase AgenteMinimax.java en Java. Debe contar con: 1. Contadores nodosEvaluadosConPoda y nodosEvaluadosSinPoda, con métodos reiniciarContadores() para reiniciarlos. 2. Un método que encuentre la mejor jugada para el jugador. Inicializa el mejorMovimiento por seguridad con el primer elemento de tablero.getMovimientosLegales(jugador) (si no está vacío) para evitar retornos nulos ante máxima penalización. Usa recursión minimax para obtener la puntuación de cada hijo. 3. El método minimax recursivo: - Caso base: profundidad == 0 o tablero.esTerminal() -> retorna EvaluadorHeuristico.evaluar para el jugadorMaximizador. - Si el jugador activo no tiene jugadas válidas (Pass), salta el turno y llama recursivamente a minimax con (!esMaximizador), reduciendo la profundidad en 1. - Implementa las ramas de maximizador y minimizador actualizando alfa y beta. Si usarPoda es true, ejecuta la poda tradicional (si beta <= alfa, romper bucle).
Resultado / cómo se usó	Proporcionó el motor de toma de decisiones parametrizado. Esto permite que en el modo Espectador (IA vs. IA) cada agente calcule su movimiento optimizando para su respectivo color (IA 1 maximizando para Negras e IA 2 para Blancas), evitando que jueguen con el mismo color.
Validación / ajuste del equipo	Validamos la eficacia de la poda y la independencia de color: al enfrentar a IA 1 (Negras, Prof. 3) contra IA 2 (Blancas, Prof. 5) en modo espectador, cada una jugó en su turno buscando su propio beneficio de color, reportando métricas de nodos de forma independiente.

Campo	Descripción
Nº	P-05
Objetivo del prompt	Estructurar el bucle principal de Main.java, el menú rejugable de selección de modos (incluyendo el modo espectador IA vs. IA), y anclar la visualización del tablero en la parte superior.
Herramienta / modelo de IA	Gemini 3.5 flash (high) (Antigravity).

Texto del prompt	Diseña la clase controladora Main.java en Java para Othello con las siguientes pautas: 1. Encierra la ejecución en un bucle while(true) infinito. Lo primero que debe hacer es limpiar la pantalla (\033[H\033[2J) y mostrar el menú: 1) Humano vs IA, 2) IA vs IA (Espectador), 3) Salir. 2. Si elige 1: pide seleccionar dificultad (profundidad 3, 5, 6 o personalizada). Negras es Humano, Blancas es IA. 3. Si elige 2: pide profundidad para IA 1 (Negras), profundidad para IA 2 (Blancas), y control de avance (1: Manual por ENTER, 2: Automático continuo con retardo de 1.8 segundos). Llama a <code>tablero.setEtiquetasJugadores</code> para configurar las etiquetas en modo espectador. 4. En el bucle de juego (!<code>tablero.esTerminal()</code>): - ANCLAJE DEL TABLERO: En cada refresco, limpia consola y dibuja de inmediato el tablero (<code>imprimirTablero</code>) en la parte superior. Las cabeceras de turno, coordenadas jugadas, tablas de métricas de poda y prompts se imprimen siempre debajo del tablero. - Turno Negras (Humano o IA 1): si es humano, valida coordenadas y juega con animación (pausa 1.2s). Si es IA 1, calcula jugada, anuncia coordenada de decisión, pausa 1.2s, anima la jugada, limpia pantalla, dibuja el tablero y muestra métricas. - Turno Blancas (IA o IA 2): ejecuta su turno de la misma forma que IA 1. 5. Al acabar la partida, muestra el tablero final arriba, el score con etiquetas correctas abajo y el ganador. Pide presionar ENTER para regresar al menú inicial.
Resultado / cómo se usó	Estructuró el flujo de turnos interactivo (gestionando tanto el modo Humano vs. IA como el Espectador IA vs. IA), implementando el anclaje visual (tablero fijo arriba) y la rejugabilidad mediante menús.
Validación / ajuste del equipo	Se corroboró que al terminar un duelo de IA vs. IA (o Humano vs. IA) se regrese correctamente al menú principal al presionar ENTER. También se validó que en modo automático las IAs jueguen fluidamente con el retardo de 1.8 segundos entre turnos.

C. Referencias Bibliográficas

- Knuth, D. E., & Moore, R. W. (1975). An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4), 293-326. [https://doi.org/10.1016/0004-3702\(75\)90019-3](https://doi.org/10.1016/0004-3702(75)90019-3)
- Norvig, P., & Russell, S. (2020). *Artificial Intelligence: A Modern Approach (4th ed.)*. Pearson Education.
- Rosenbloom, P. S. (1982). A world-championship-level Othello program. *Artificial Intelligence*, 19(3), 279-320.
- Shannon, C. (1950). "Programming a computer for playing chess", *Philosophical Magazine Ser. 7*, vol. 41.